



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Introduction to Flutter Architecture and Widgets

Dr. Annu Sharma

Department of Computer Applications

Mobile App Development

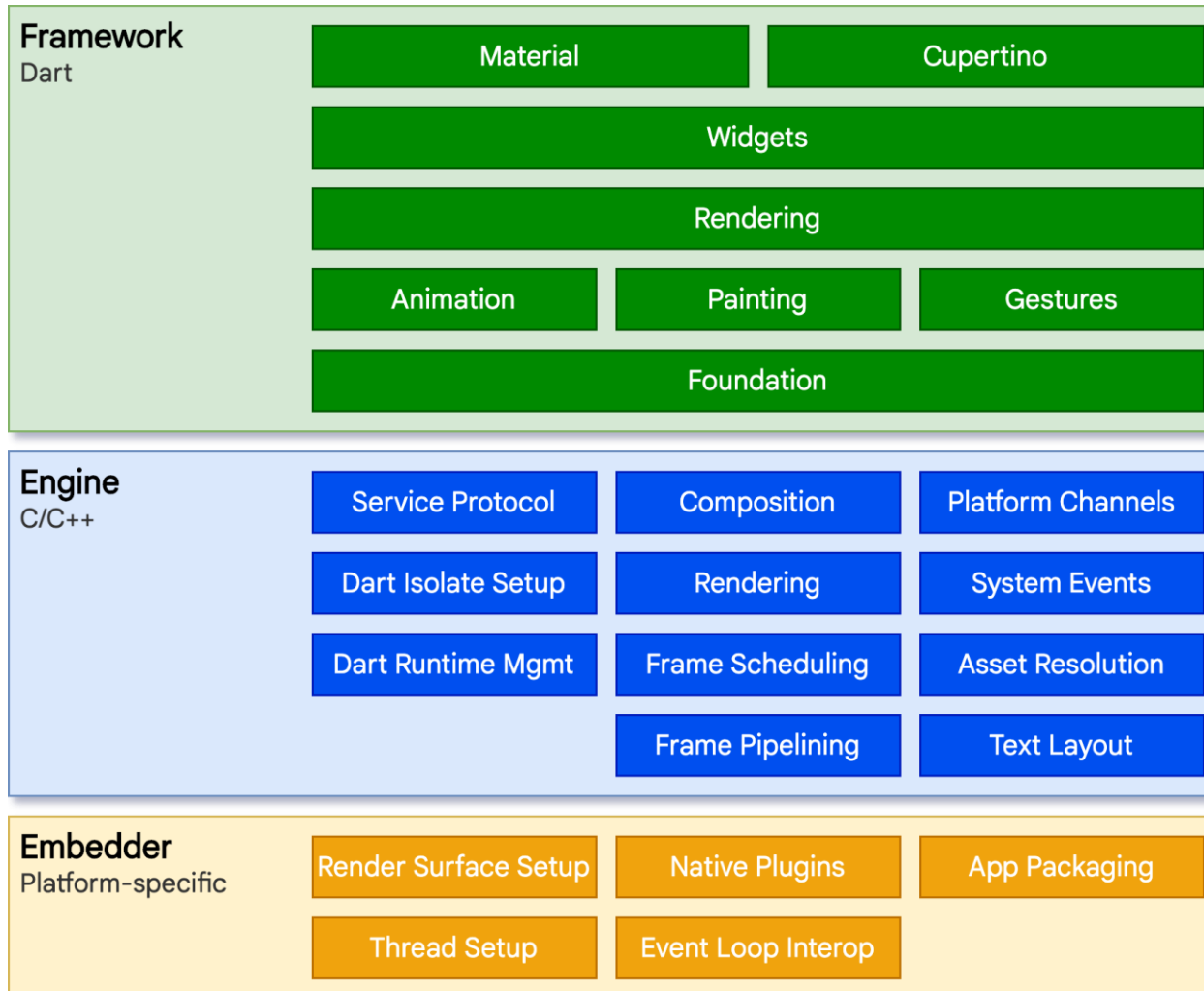
Introduction to Flutter Widgets



- In Flutter, **widgets** are the building blocks of the user interface.
- Every visual, structural, and interactive component in a Flutter app is a widget, including elements like buttons, images, layouts, etc.
- Widgets in Flutter are classified into **stateless** and **stateful**.
- Widgets form a widget tree, where parent widgets hold child widgets, creating the structure of the app's UI.

Mobile App Development

Flutter Architecture



Key Layers of Flutter Architecture:

1. Flutter Framework (Dart Code)
2. Flutter Engine (C/C++ Code)
3. Embedder Layer
4. Underlying Platform (iOS, Android, Web, etc.)

- Basic **foundational** classes, and building block services such as **animation**, **painting**, and **gestures** that offer commonly used abstractions over the underlying foundation.
- The **rendering layer** provides an abstraction for dealing with layout. With this layer, you can build a tree of renderable objects. You can manipulate these objects dynamically, with the tree automatically updating the layout to reflect your changes.
- The **widgets layer** is a composition abstraction. Each render object in the rendering layer has a corresponding class in the widgets layer. In addition, the widgets layer allows you to define combinations of classes that you can reuse. This is the layer at which the reactive programming model is introduced.
- The **Material** and **Cupertino** libraries offer comprehensive sets of controls that use the widget layer's composition primitives to implement the Material or iOS design languages.

Mobile App Development

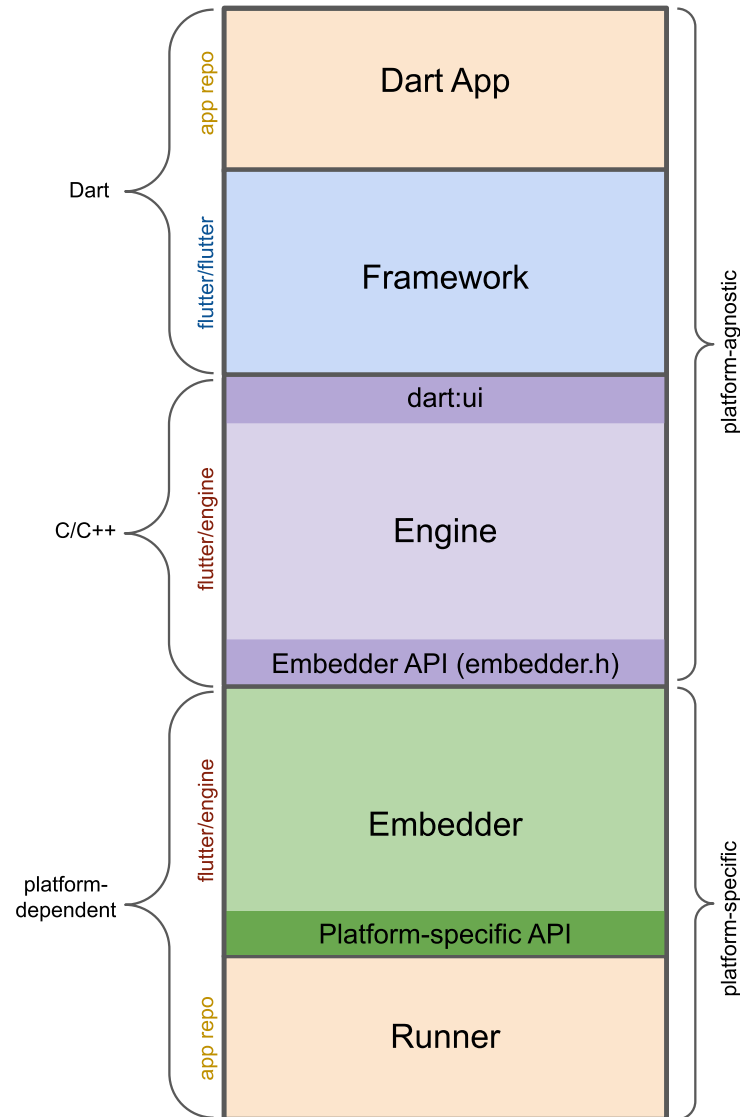
Flutter Architecture - Summary



- Flutter's architecture is designed to be modular, fast, and reactive.
- It consists of the **Flutter framework** (for UI design and business logic), the **Flutter engine** (for rendering and input handling), and the **embedder layer** (for platform-specific integration).
- Together, these layers provide a powerful and flexible cross-platform development environment capable of delivering high-performance apps with a unified codebase.

Mobile App Development

Flutter App Anatomy



A. Dart App

- Composes widgets into the desired UI.
- Implements business logic.
- Owned by app developer.

B. Framework

- Provides higher-level API to build high-quality apps (for example, widgets, hit-testing, gesture detection, accessibility, text input).
- Composites the app's widget tree into a scene.

C. Engine

- Provides low-level implementation of Flutter's core APIs (for example, graphics, text layout, Dart runtime).
- Exposes its functionality to the framework using the **dart:ui API**.
- Integrates with a specific platform using the Engine's **Embedder API**.

D. Embedder

- Coordinates with the underlying operating system for access to services like rendering surfaces, accessibility, and input.
- Manages the event loop.
- Exposes **platform-specific API** to integrate the Embedder into apps.

E. Runner

- Composes the pieces exposed by the platform-specific API of the Embedder into an app package runnable on the target platform.
- Part of app template generated by flutter create, owned by app developer.

- **Stateless Widgets:** These are immutable widgets, meaning their configuration and properties cannot change after they are created. They are used for static content.
- **Stateful Widgets:** These widgets can maintain and update their state during their lifecycle. When the state changes, the widget rebuilds its UI.

Mobile App Development

Stateful v/s Stateless Widgets

Aspect	Stateless Widgets	Stateful Widgets
Mutability	Immutable: Cannot change once built	Mutable: Can change dynamically during the app's lifecycle
State Management	No internal state management	Manages state using an internal <code>State</code> object
Lifecycle	Stateless widgets are rebuilt only when parent widget rebuilds	Stateful widgets can rebuild themselves upon state changes
Rebuild Trigger	Only rebuilt if the parent widget changes or the widget tree is rebuilt	Rebuilt when <code>setState()</code> is called or when an event triggers a change
Use Cases	For static or unchanging UI components	For dynamic or interactive UI components
Performance	More efficient due to lack of state management	Less efficient as they need to manage and monitor state
Examples	<code>Text</code> , <code>Icon</code> , <code>Image</code> , <code>Container</code> , <code>Row</code> , <code>Column</code>	<code>TextField</code> , <code>Checkbox</code> , <code>Slider</code> , Forms, Lists with dynamic content

Mobile App Development

Stateful and Stateless Widgets



When to Use Stateless Widgets:

- **Static UI:** When you need to display content that does not change during the lifecycle of the widget.
- **Simple components:** For displaying text, images, or arranging elements on the screen without requiring updates.

When to Use Stateful Widgets:

- **Interactive Components:** When the widget's appearance or behaviour needs to change based on user interactions.
- **Dynamic Data:** When the widget needs to update its UI based on changes in its data, such as user inputs, animations, or real-time data fetching.

Mobile App Development

Hot Reload and Hot Restart

- **Hot Reload:** When making changes in your code, you can use Hot Reload to see the updates instantly without losing the current state of the app. It is extremely fast and useful during UI development.
- **Hot Restart:** Hot Restart, on the other hand, resets the entire app, losing the state but applying all changes. It is more comprehensive than Hot Reload but slower.



Mobile App Development

Hot Reload v/s Hot Restart

Feature	Hot Reload	Hot Restart
Definition	Injects updated code into the running app	Rebuilds the entire app from scratch
State Preservation	Preserves the app's state	Resets the app's state
Rebuild Scope	Rebuilds only the widget tree that changed	Rebuilds the entire app, starting from <code>main()</code>
Speed	Extremely fast (less than a second)	Slower than Hot Reload, but faster than a full restart
Use Cases	UI updates, layout tweaks, quick changes	State-related changes, global variables, resetting app state
Typical Scenarios	Changing UI elements, modifying layout	Changing global variables, resetting the app, main method changes



THANK YOU

Ms. Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Introduction to Flutter Architecture and Widgets

Dr. Annu Sharma

Department of Computer Applications

Mobile App Development

Creating Flutter App

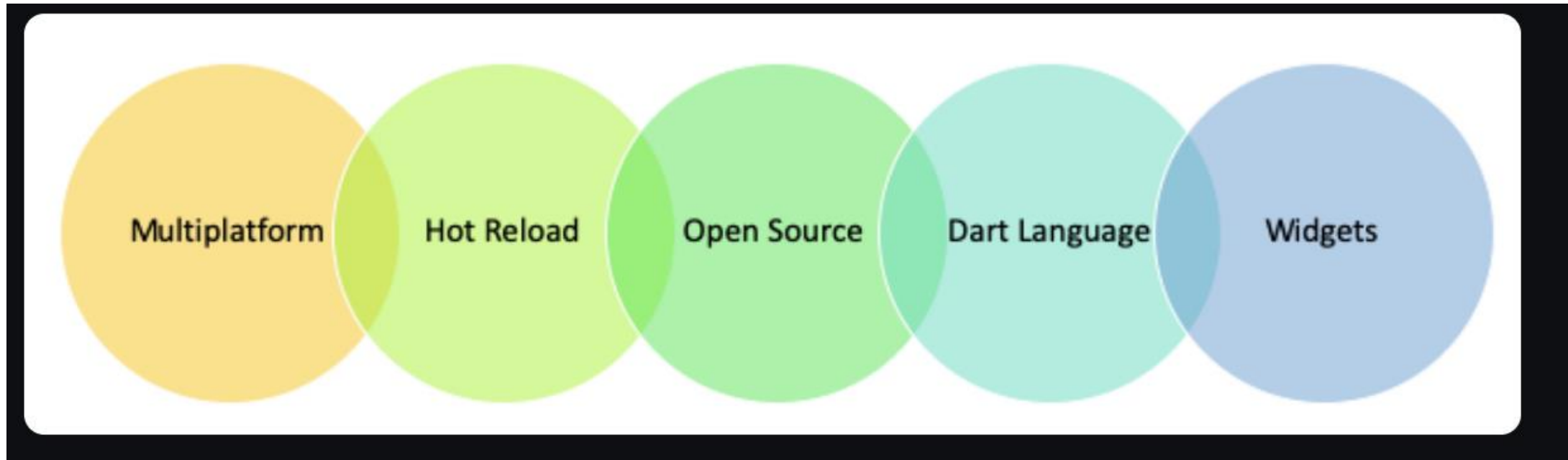


- Flutter is a software development toolkit from Google for building cross-platform apps.
- Flutter apps consist of a series of packages, plugins and widgets — but that's not all. Flutter is a process, a philosophy and a community as well.
- It's also the easiest way to get an app up and running on any one platform, let alone multiple.
- Its declarative, widget-based UI structure, first class support for reactive programming, cross platform abstractions and virtual machine.
- It allows for hot reloading of code changes mean you can be way more productive than you thought possible.

Mobile App Development

Creating Flutter App

Flutter is *not* a language. Flutter uses **Dart** as its programming language. it's an object-oriented language.



Flutter Framework

UI Theme - Cupertino / Material

Widgets - Styling, Text, Controls

Rendering

Foundation - painting, animation, gestures

The Flutter framework consists of several sublayers:

- At the top is the **UI theme**, which uses either the Material (Android) or Cupertino (iOS) design language. This affects how the controls appear, allowing you to make your app look just like a native one.
- The **widget layer** is where you'll spend the bulk of your UI programming time. This is where you compose design and interactive elements to make up the app.
- Beneath the widgets layer is the **rendering layer**, which is the abstraction for building a layout.
- The **foundation layer** provides basic building blocks, like animations and gestures, that build up the higher layers.

Install Prerequisites

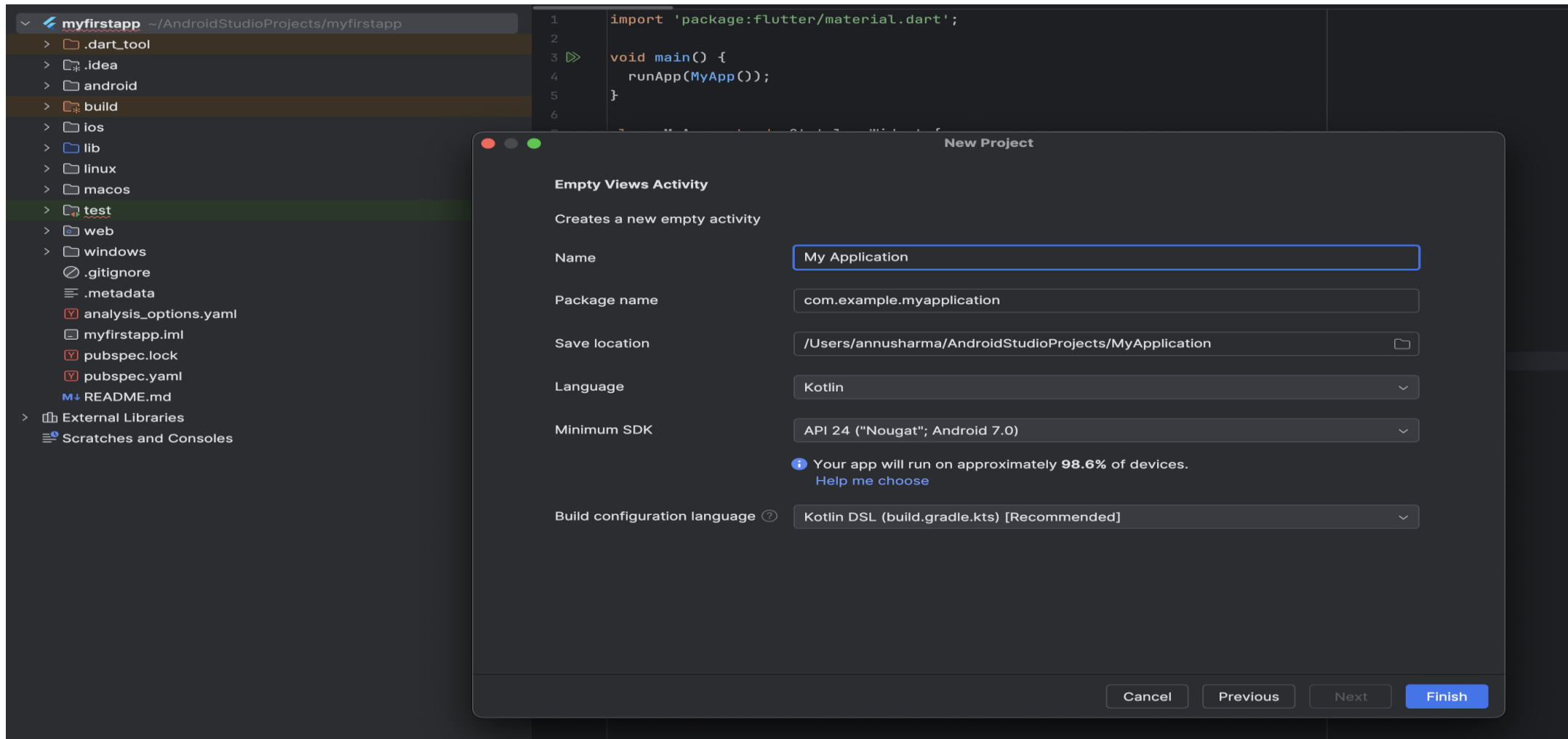
- **Install Flutter SDK** → flutter.dev
- **Install Android Studio** → Make sure the **Flutter** and **Dart** plugins are installed.
 - Go to **File** → **Settings** → **Plugins** → **Search Flutter** → **Install** (it auto-installs Dart too).

2. Create a New Flutter Project

1. Open **Android Studio**.
2. Click **New Project** → Select **Flutter Application**.
3. Configure:
 1. **Project name** → e.g., my_first_app
 2. **Flutter SDK path** → Browse to Flutter SDK folder
 3. **Project location** → Choose folder
4. Click **Finish**

Mobile App Development

Creating Flutter App

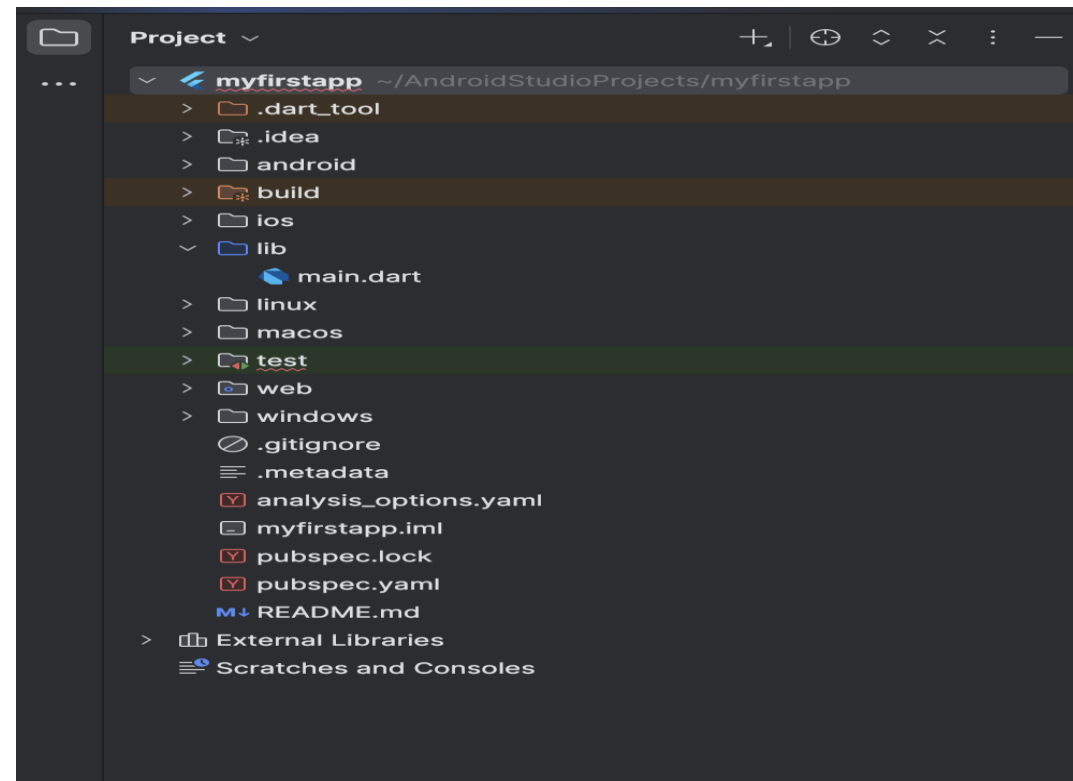


3. Explore Project Structure

- lib/main.dart → Main file where Flutter code is written.
- pubspec.yaml → Manage dependencies, fonts, and assets.
- android/, ios/, web/ → Platform-specific folders.

Mobile App Development

Creating Flutter App



Mobile App Development

Creating Flutter App

4. Write a Basic Program

Open **lib/main.dart** and replace code with:

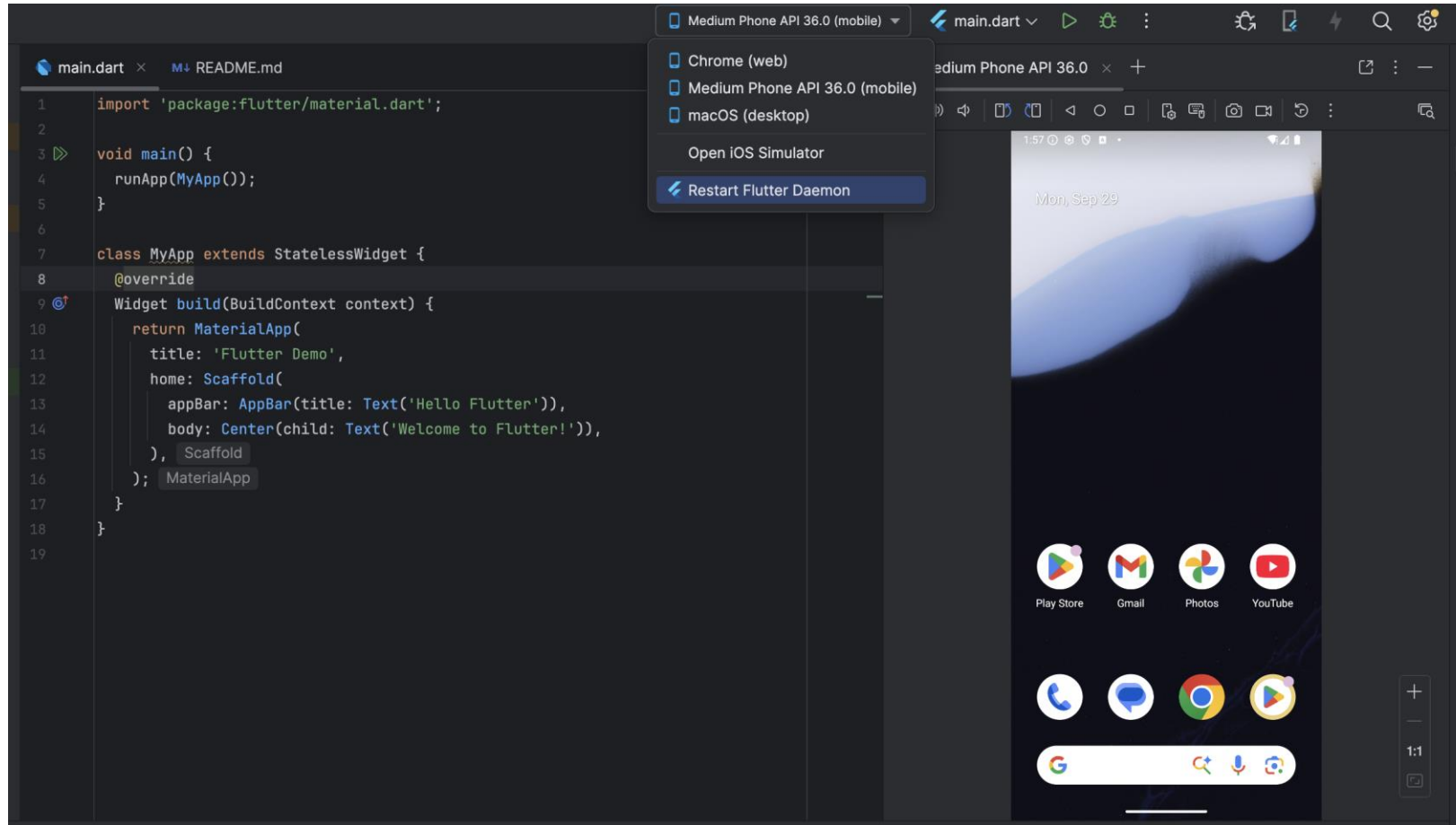
```
main.dart x M README.md
1 import 'package:flutter/material.dart';
2
3 void main() {
4   runApp(MyApp());
5 }
6
7 class MyApp extends StatelessWidget {
8   @override
9   Widget build(BuildContext context) {
10     return MaterialApp(
11       title: 'Flutter Demo',
12       home: Scaffold(
13         appBar: AppBar(title: Text('Hello Flutter')),
14         body: Center(child: Text('Welcome to Flutter!')),
15       ), Scaffold
16     ); MaterialApp
17   }
18 }
19
```

5. Run the Program

1. Connect a **physical Android device** (with USB debugging ON) or start an **emulator** from Android Studio.
2. Click the **green play**  **button** (Run).
3. The app will build and launch showing “**Welcome to Flutter!**” on screen.

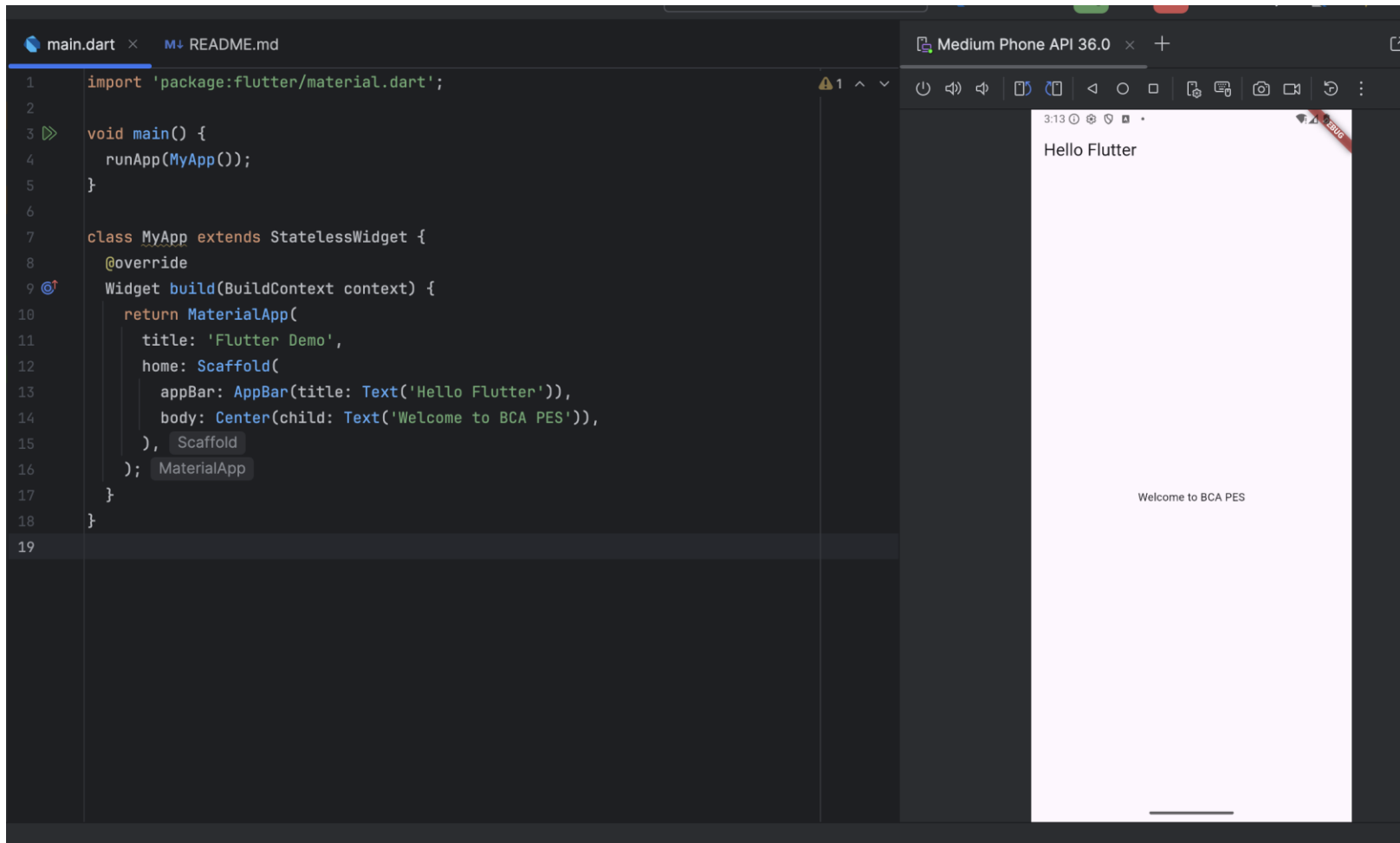
Mobile App Development

Creating Flutter App



Mobile App Development

Creating Flutter App



The screenshot displays an IDE with two panels. The left panel shows the `main.dart` file with the following Dart code:

```
1 import 'package:flutter/material.dart';
2
3 void main() {
4   runApp(MyApp());
5 }
6
7 class MyApp extends StatelessWidget {
8   @override
9   Widget build(BuildContext context) {
10    return MaterialApp(
11      title: 'Flutter Demo',
12      home: Scaffold(
13        appBar: AppBar(title: Text('Hello Flutter')),
14        body: Center(child: Text('Welcome to BCA PES')),
15      ),
16    );
17  }
18 }
19
```

The right panel shows a mobile emulator running the application. The emulator's status bar at the top indicates the time is 3:13 and the battery level is 33%. The app's interface consists of a white background with the text "Hello Flutter" at the top and "Welcome to BCA PES" centered below it. A red "BUG" banner is visible in the top right corner of the emulator screen.

Which file in a Flutter project contains the main entry point of the app?

- a) pubspec.yaml
- b) main.dart
- c) gradle.build
- d) manifest.xml

2. Which command/function starts the execution of a Flutter application?

- a) mainApp()
- b) runApp()
- c) startApp()
- d) launchApp()

3. In a basic Flutter app, which widget is commonly used to set up the visual structure with AppBar, Body, etc.?

- a) Container
- b) Scaffold
- c) Column
- d) Stack

Mobile App Development

Creating Flutter App



Which file in a Flutter project contains the main entry point of the app?

Answer: b) main.dart

2. Which command/function starts the execution of a Flutter application?

Answer: b) runApp()

3. In a basic Flutter app, which widget is commonly used to set up the visual structure with AppBar, Body, etc.?

Answer: b) Scaffold



THANK YOU

Ms. Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Introduction to Material App and Scaffold Widget

Dr. Annu Sharma

Department of Computer Applications

Mobile App Development

Material Widgets

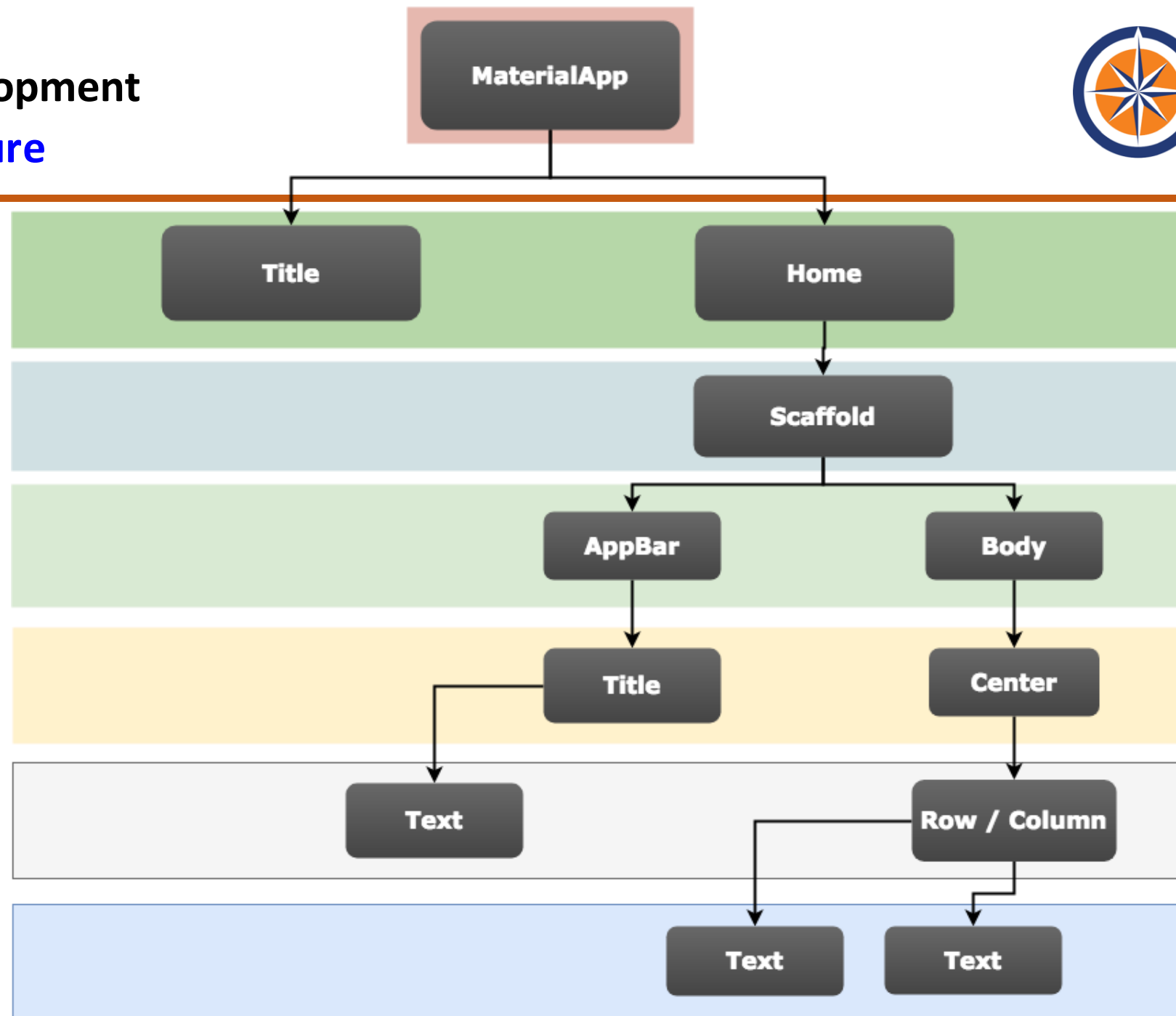


- Scaffold
- AppBar
- BottomNavigationBar
- TabBar
- TabBarView
- ListTile
- RaisedButton
- FloatingActionButton
- FlatButton
- IconButton
- DropdownButton
- PopupMenuButton
- ButtonBar
- TextField
- Checkbox
- Radio
- Switch
- Slider
- Date & Time Pickers
- SimpleDialog
- AlertDialog

- In Flutter, a widget can be created by composing one or more widgets.
- To compose multiple widgets into a single widget, **Flutter** provides large number of widgets with layout feature.
 - a. **Container** – A rectangular box decorated using *BoxDecoration* widgets with background, border and shadow.
 - b. **Center** – Center its child widget.
 - c. **Row** – Arrange its children in the horizontal direction.
 - d. **Column** – Arrange its children in the vertical direction.
 - e. **Stack** – Arrange one above the another.

Mobile App Development

Flutter App Structure



Mobile App Development

MaterialApp Widget



- **MaterialApp** is a Flutter widget that wraps the entire app in a Material Design style.
- It serves as the top-level widget for an app, providing functionality like routing, themes, and localization.
- By using **MaterialApp**, you can ensure that the app adheres to Google's Material Design guidelines. Key properties include:
 - (i) **home**: The widget that is displayed first when the app starts.
 - (ii) **routes**: A map of route names to widgets for navigation.
 - (ii) **theme**: To define the app's visual theme.

Mobile App Development

Scaffold Widget



- The **Scaffold** widget is a key part of any Material Design app.
- It provides a basic structure like a background, app bar, floating action button, bottom navigation bar, and drawers.
- **Scaffold** act as a canvas on which you place your widgets to form the user interface. Key properties include:
 - (i) **appBar**: Displays an app bar with titles and actions.
 - (ii) **body**: Contains the primary content of the screen.
 - (iii) **floatingActionButton**: A button typically used for primary actions on the screen.

Feature	MaterialApp	Scaffold
Definition	Root widget of a Flutter app that provides Material Design structure and theme for the entire app	Provides the basic visual layout structure for a screen (page) like AppBar, Body, FAB, Drawer, BottomNavigationBar
Purpose	Sets up the overall app environment (theme, routes, title, navigator, localization, etc.)	Creates individual screens/pages inside the app
Hierarchy	Usually at the top level inside runApp()	Usually inside home or body of MaterialApp

Mobile App Development

Scaffold Widget



Feature	MaterialApp	Scaffold
Hierarchy	Usually at the top level inside runApp()	Usually inside home or body of MaterialApp
Provides	- App theme- App title- Routing/navigation- Localization- Material design support	- AppBar- Body- FloatingActionButton- Drawer- BottomNavigationBar- SnackBar
Example Usage	<pre>dart MaterialApp(title: 'My App', theme: ThemeData(), home: MyHomePage(),)</pre>	<pre>dart Scaffold(appBar: AppBar(title: Text('Home')), body: Center(child: Text('Hello')),)</pre>
Scope	Entire app	Single screen/page

Mobile App Development

Scaffold Widget

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    title: 'Material vs Scaffold Example',
    home: Scaffold(
      appBar: AppBar(title: Text('Home Page')),
      body: Center(child: Text('Hello Flutter!')),
      floatingActionButton: FloatingActionButton(
        onPressed: () {},
        child: Icon(Icons.add),
      ),
    ),
  ));
}
```

Compiled By : Annu Sharma - Department of Computer Applications

MaterialApp wraps the whole app.

Scaffold creates a screen with AppBar, Body, and FloatingActionButton.

Mobile App Development

Material & Scaffold Widget



1. Which widget is used as the root of a Flutter app that uses Material Design?

- A. CupertinoApp
- B. MaterialApp
- C. Scaffold
- D. AppBar

Answer: B. MaterialApp

2. What is the main purpose of the MaterialApp widget?

- A. To provide basic routing and theme configuration
- B. To build a layout structure
- C. To add icons to the app
- D. To handle database operations

Answer: A. To provide basic routing and theme configuration

3. Which of the following is NOT a property of MaterialApp?

- A. home
- B. routes
- C. theme
- D. body

Answer: D. body

Mobile App Development

Material & Scaffold Widget



6. Which widget provides the basic visual layout structure in a Material Design app?

- A. Column
- B. Container
- C. Scaffold
- D. AppBar

Answer: C. Scaffold

7. Which property of Scaffold is used to display content in the main body area?

- A. appBar
- B. drawer
- C. body
- D. floatingActionButton

Answer: C. body



THANK YOU

Ms. Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOPs and Flutter Widgets

Radio Buttons

Dr. Annu Sharma

Department of Computer Applications

Radio Button

- In Flutter, a RadioButton is represented by the Radio widget.
- It's used to create a single choice among multiple options.
- Typically, it's used within a ListTile or alongside labels to form a set of mutually exclusive choices
- e.g., selecting a gender or a favorite color.

Radio Button Key Concepts:

- **value:** The value that represents each radio button.
- **GroupValue:** The shared value among all radio buttons in the group, determining which one is currently selected.
- **onChanged:** A callback that updates the state when the user selects a radio button.

1. Stateful Widget:

```
class RadioButtonExample extends StatefulWidget {  
  @override  
  _RadioButtonExampleState createState() =>  
  _RadioButtonExampleState();  
}
```

Description

Here `RadioButtonExample` is a custom widget that extends `StatefulWidget`. `createState()` is a method that returns an instance of the state class `_RadioButtonExampleState`, which will manage the widget's state.

2. State Class:

```
class _RadioButtonExampleState extends State<RadioButtonExample> {  
  
  int _selectedValue = 1;
```

Description

`_RadioButtonExampleState` is the actual state class for `RadioButtonExample`. This is where the widget's logic and state are defined.

`_selectedValue`: This is a private variable (denoted by the underscore `_`), and it holds the currently selected radio button's value. The default value is set to 1, meaning the first option will be selected by default when the widget is first rendered.

3. State Management:

Whenever a radio button is selected, the `_selectedValue` is updated using **`setState()`**. This triggers the widget to rebuild, showing the newly selected option.

4. Build Method

This part of the code defines the build method, which describes the user interface of the widget and is responsible for creating and returning the layout of the radio buttons. Let's break it down:

```
@override  
Widget build(BuildContext context) {  
  return Column(  
    children: <Widget>[
```

- The build method is where the UI is constructed.
- Column is a widget that arranges its children vertically (in this case, the radio buttons).
- children is a list of widgets that will be displayed inside the Column.
- **ListTile:** This is a convenient way to organize the radio button and the text (Option 1) in a single horizontal row.
- **title: Text('Option 1'):** Displays the text "Option 1" next to the radio button.
- **leading: Radio():** Places a radio button before the text, in the leading position (left side by default).

- **ListTile(**
title: Text('Option 1'),
leading: Radio(
value: 1,
groupValue: _selectedValue,
onChanged: (int? value) {
setState(() {
_selectedValue = value!;
});
},
),

Radio Widget:

`value: 1`: This radio button has the value 1. If selected, `_selectedValue` will be updated to 1.

`groupValue: _selectedValue`: All radio buttons in the same group share the same `groupValue`. Here, it compares the current `_selectedValue` with the radio button's value. If they match, this button will be shown as selected.

onChanged:

This is a callback that gets triggered when the user taps the radio button.

The value parameter contains the value of the selected radio button (in this case, 1). Inside `setState()`, `_selectedValue` is updated with the selected value. This causes the widget to rebuild and reflect the new state (i.e., it will now show the newly selected radio button).

Mobile App Development

Dart OOPs Concept



Radio Button Example

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Select the course using
Radio Button'),
        ),
```

```
body: RadioButtonExample(),
      ), );
  }
}

class RadioButtonExample extends StatefulWidget {
  @override
  _RadioButtonExampleState createState() =>
    _RadioButtonExampleState();
}

class _RadioButtonExampleState extends
State<RadioButtonExample> {
  int _selectedValue = 2;
  @override
  Widget build(BuildContext context) {
    return Column(
      children: <Widget>[
```


Radio Button Example

```
ListTile(  
  title: Text('MCA'),  
  leading: Radio(  
    value: 1,  
    groupValue: _selectedValue,  
    onChanged: (int? value) {  
      setState(() {  
        _selectedValue = value!  
      });  
    },  
  ),  
),
```

```
ListTile(  
  title: Text('BCA'),  
  leading: Radio(  
    value: 2,  
    groupValue: _selectedValue,  
    onChanged: (int? value) {  
      setState(() {  
        _selectedValue = value!  
      });  
    },  
  ),  
),
```

Similarly you can go for creating as many lists as you want using Listtile

Mobile App Development

Dart OOPs Concept

```
ListTile(  
  title: Text('BBA'),  
  leading: Radio(  
    value: 3,  
    groupValue: _selectedValue,  
    onChanged: (int? value) {  
      setState(() {  
        _selectedValue = value!  
      });  
    },  
  ),  
),  
],  
);}}
```

Output

Select the course using Radio Button

- ☐ MCA
- ☒ BCA
- ☐ BBA

1. Which widget is used to create a single radio button in Flutter?

- A. Checkbox
- B. Switch
- C. Radio
- D. ToggleButton

Answer: C. Radio

2. Which property of the `Radio` widget determines which option is currently selected?

- A. `activeColor`
- B. `groupValue`
- C. `value`
- D. `onChanged`

Answer: B. `groupValue`

3. What is the data type of the `value` and `groupValue` properties in a `Radio` widget?

- A. `int` only
- B. `String` only
- C. Any data type (generic type `<T>`)
- D. `bool`

Answer: C. Any data type (generic type `<T>`)

Challenge

Create a Radio Button Program in flutter with title as Select the Elective-Course of your choice add three Radio buttons as : Mobile App Development, Graphic Design, and Sw Testing. Set the default value as MAD.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Widgets and Navigations

Checkboxes

Dr. Annu Sharma
Department of Computer Applications

Checkboxes Button

- Checkboxes are typically used to represent a binary state (checked or unchecked) and are interactive.

Explanation

- **CheckboxListTile:** This is a common widget used for checkboxes in a list. It combines a Checkbox with a label (Text) and handles the state management internally. It's commonly used in forms, settings screens, etc.

Checkboxes Concepts:

- **value:** This holds the current state of the checkbox (true or false).
- **title:** The text that appears next to the checkbox.
- **onChanged:** This is a callback that is called when the checkbox is tapped. It changes the state of the checkbox.

State Management: The checkbox state (isChecked) is managed using the StatefulWidget's setState method.

Mobile App Development

Widgets and Navigations



```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomePage(), ); } }
```

Checkboxes

```
class HomePage extends StatefulWidget {  
  @override  
  _HomePageState createState() => _HomePageState();  
}  
  
class _HomePageState extends State<HomePage> {  
  bool? value = false; // Initialize as nullable bool for null safety  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('PES UNIVERSITY'),  
        backgroundColor: Colors.greenAccent[400],
```

Mobile App Development

Widgets and Navigations



```
        leading: IconButton(
          icon: const Icon(Icons.menu),
          tooltip: 'Menu',
          onPressed: () {

            },
          ),
        ),
        body: Center(
          child: Card(
            child: Padding(
              padding: const EdgeInsets.all(15.0),
              child: SizedBox(
                width: 430,
                height: 700,
                child: Column(
```

Mobile App Development

Widgets and Navigations



```
child: Column(  
  children: [  
    Text(  
      ' WELCOME TO COMPUTER  
APPLICATIONS DEPARTMENT',  
      style: TextStyle(  
        color: Colors.greenAccent[400],  
        fontSize: 30),  
    ),  
    const SizedBox(height: 10),  
    Row(  
      children: <Widget>[  
        const SizedBox(width: 10),  
        const Text(  
          'CLICK BCA COURSE',  
          style: TextStyle(fontSize: 17.0),  
        ),  
        const SizedBox(width: 10),  
        Checkbox(  

```

Mobile App Development

Widgets and Navigations



```
Checkbox(  
    tristate: true, // Example with tristate  
    value: value,  
    onChanged: (bool? newValue) {  
        setState(() {  
            value = newValue;  
        });  
    },  
),  
],  
,  
],  
,  
],  
,  
,  
,  
,  
,  
,  
,  
);  
}  
}
```

Mobile App Development

Widgets and Navigations



1. Which widget is used to create a checkbox in Flutter?

- A. Radio
- B. Switch
- C. Checkbox
- D. ToggleButtons

Answer: C. Checkbox

2. What is the data type of the `value` property in a `Checkbox` widget?

- A. int
- B. String
- C. bool
- D. double

Answer: C. bool

3. If the value of a `Checkbox` is null and `tristate` is false, what happens?

- A. Checkbox is disabled
- B. Checkbox shows a dash
- C. Checkbox throws an error
- D. Checkbox defaults to unchecked (false)

Answer: C. Checkbox throws an error



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOPs and Flutter Widgets

List View

Dr. Annu Sharma

Department of Computer Applications

List View

Introduction to ListView

- A scrollable list of widgets
- - Perfect for displaying data in a single column

Use cases:

- Chat apps, item lists, feeds, product listings, etc.

List View

- ListView is a scrollable list of widgets arranged linearly.
- It displays its children one after another in the scroll direction i.e, vertical or horizontal.
- There are different types of ListViews

Types of ListView

1. **ListView** – Standard, single-direction scrollable list.
2. **ListView.builder** – Efficient for dynamically creating large lists.
3. **ListView.separated** – Allows adding separators between items.
4. **ListView.custom** – Offers flexibility with a custom sliver list.

List View Description

MaterialApp: Sets up the app with Material design and routes.

Scaffold: Provides a basic app layout with an AppBar and a body.

AppBar: A top bar with a title for the app.

ListView: A scrollable list container.

ListTile: A widget that creates a basic, consistent layout for each item in the list. Here, each ListTile has an icon (leading) and a title (title).

- The build method is where the UI is constructed.
- Column is a widget that arranges its children vertically (in this case, the List View).
- children widgets that will contain **ListTile**
- **ListTile**: This is a convenient way to organize the List View and the text Option in a single horizontal row.
- **leading**: Icon(Icons.map): Places a Icon of map before the text, in the leading position (left side by default).
- **title**: **Text(' MAP')**: Displays the text Option next to the ICON.

List View

```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Basic ListView',
      home: Scaffold(
```

List View

```
    appBar: AppBar(
      title: Text('Basic ListView
Example'), ),
    body: ListView(
      children: <Widget>[
        ListTile(
          leading: Icon(Icons.map),
          title: Text('Map'),
        ),
        ListTile(
```

List View

```
ListTile(  
  leading: Icon(Icons.photo),  
  title: Text('Photos'), ),  
ListTile(  
  leading: Icon(Icons.phone),  
  title: Text('Phone'), ),  
ListTile(  
  leading: Icon(Icons.contacts),
```

List View

```
  leading: Icon(Icons.contacts),  
  title: Text('Contacts'),  
  ),  
],  
),  
),  
);  
}  
}
```

Output List View

Basic ListView Example



Map



Photos



Phone



Contacts

`_showSnackBar` in List View

```
body: ListView(  
  children: <Widget>[  
    ListTile(  
      leading: Icon(Icons.map),  
      title: Text('Map'),  
      onTap: ()  
      {  
        _showSnackBar(context, 'Map selected');  
      },  
    ),  
  ],  
)
```

_showSnackBar Definition in List View

```
void _showSnackBar(BuildContext context, String message)
{
    final snackBar = SnackBar(content: Text(message));

    ScaffoldMessenger.of(context).showSnackBar(snackBar);
}
```

Mobile App Development

Flutter Widgets



1. Which ListView constructor is used for creating a list with a large or infinite number of items?

- A. ListView()
- B. ListView.builder()
- C. ListView.separated()
- D. ListView.custom()

Answer: B. ListView.builder()

2. By default, the ListView scrolls in which direction?

- A. Horizontal
- B. Diagonal
- C. Vertical
- D. Both directions

Answer: C. Vertical

3. Which ListView constructor is used for creating a list with a large or infinite number of items?

- A. ListView()
- B. ListView.builder()
- C. ListView.separated()
- D. ListView.custom()

Answer: B. ListView.builder()

Challenge

Create a List View Program in flutter with title as Select the mode of payment choice add three Icons of your choice in list.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Layout Widgets

- Container Widget
- Row and Column

Dr. Annu Sharma

Department of Computer Applications

- In Flutter, a Container widget is one of the most commonly used widgets.
- It's a versatile widget that allows you to decorate, position, and size child widgets.
- The Container can hold a **single child** widget and apply various styles like padding, margin, alignment, decoration, and more.

- In Flutter, a Container widget is one of the most commonly used widgets.
- It's a versatile widget that allows you to decorate, position, and size child widgets.
- The Container can hold a **single child** widget and apply various styles like padding, margin, alignment, decoration, and more.

It used for layout, styling, and positioning.

- Can hold a single child widget
- Supports padding, margin, alignment, decoration
- Often used as a wrapper for other widgets

Mobile App Development

Container Widget



Syntax : Container Widget

```
Container(  
  height: 100,  
  width: 100,  
  color: Colors.blue,  
  child: Text("Hello"),  
)
```

Container Widget Properties

Property	Description
height / width	Set size of container
color	Background color
padding / margin	Inner / outer spacing
alignment	Align child widget
decoration	Add border, gradient, shadow

- a. **width & height:** Set the size of the container.
- b. **padding:** Adds space inside the container around the child.
- c. **margin:** Adds space outside the container.
- d. **alignment:** Aligns the child widget inside the container.
- e. **decoration:** Decorates the container, where you can set background color, border, border radius, shadows, etc.

The Container is a flexible widget that you can customize for various purposes in your Flutter layout.

Mobile App Development

Container Widget Example

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text("Container Example"),
          backgroundColor: Colors.redAccent,
        ),
        body: Center(
          child: Container(
            width: 400,           // Width of container
            height: 300,         // Height of container
            padding: EdgeInsets.all(20), // Inner spacing
            margin: EdgeInsets.all(15),  // Outer spacing
```

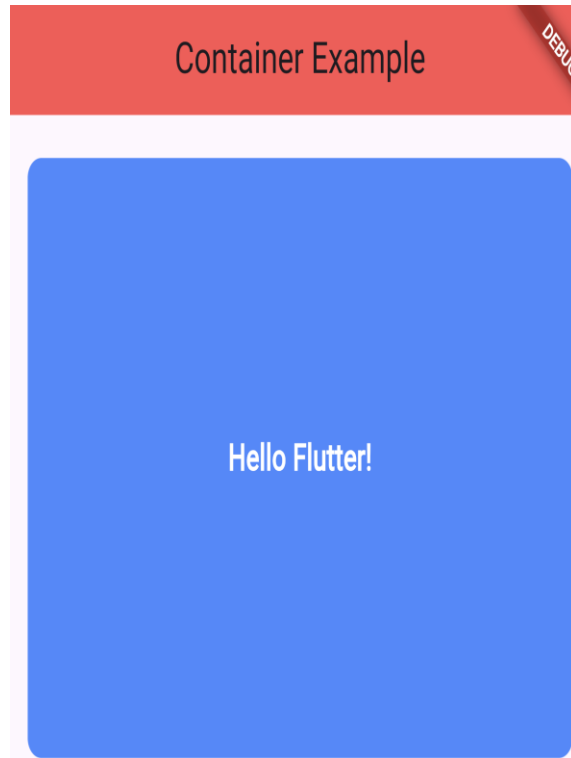
```
margin: EdgeInsets.all(15), // Outer spacing
            alignment: Alignment.center, // Centers
            decoration: BoxDecoration(
              color: Colors.blueAccent, // Background col
              borderRadius: BorderRadius.circular(10),
            ),
            child: Text(
              "Hello Flutter!",
              style: TextStyle(
                color: Colors.white,
                fontSize: 18,
                fontWeight: FontWeight.bold, ),
            ),
          ),
        ),
      );
  }
}
```

Complied By : Dr.Annu Sharma - Department of Computer Applications

Mobile App Development

Container Widget Example

Container Widget output



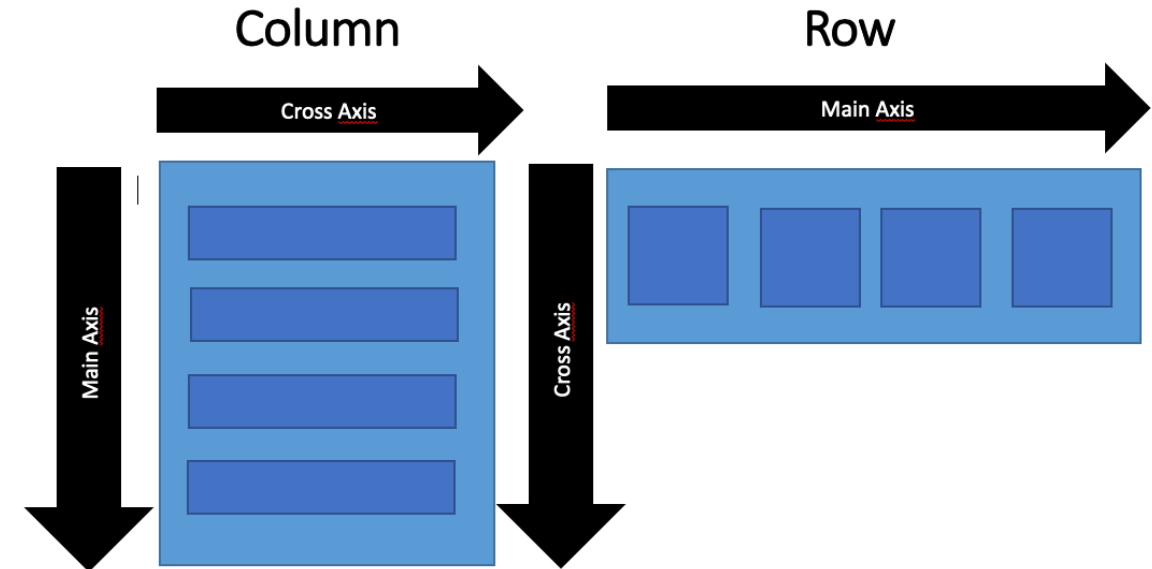
Mobile App Development

Row and Column Widgets

Row and Column are the two most important and powerful widgets in Flutter.

These widgets let you align children horizontally and vertically as per the requirement.

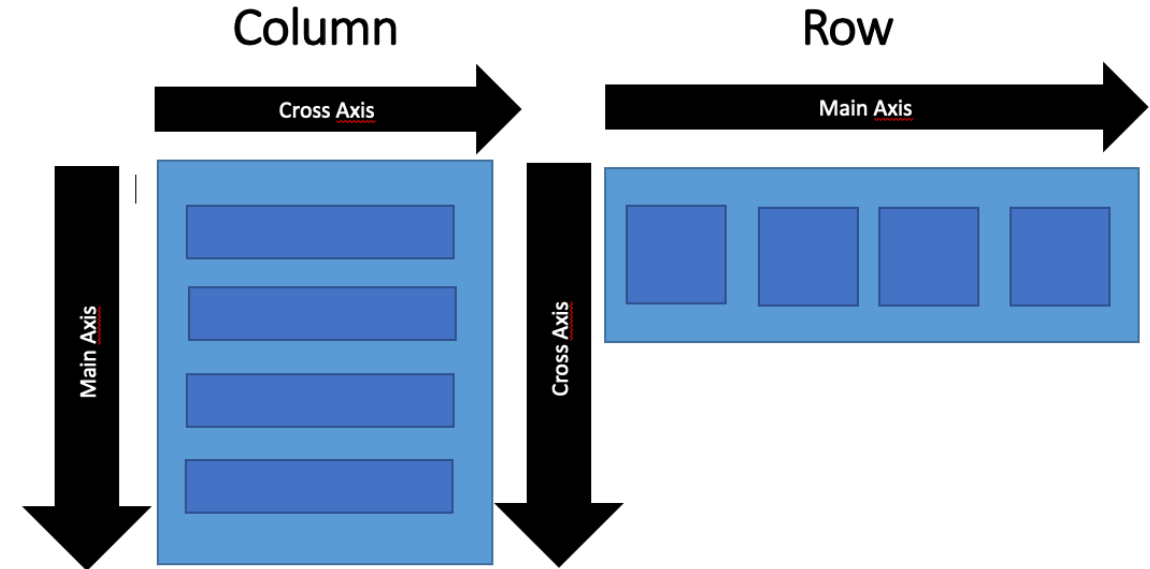
As we know that when we design any UI(User Interface) in a flutter, we need to arrange its content in the Row and Column manner so these Row and Column widgets are required when designing UI.



Mobile App Development

Row and Column Widgets

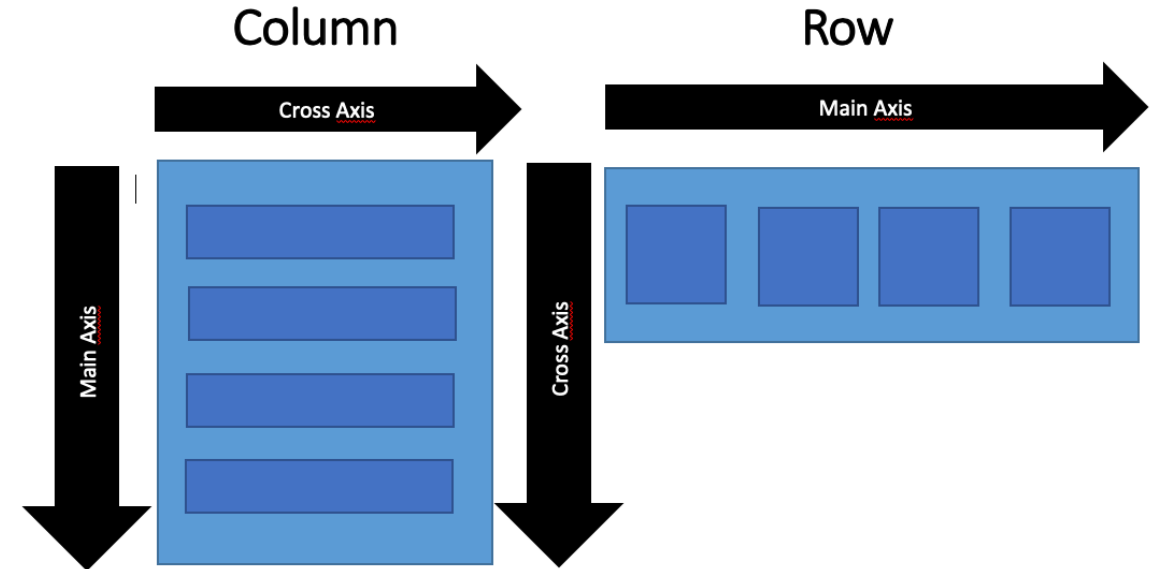
- In Flutter, **Row** and **Column** are basic layout widgets used to arrange child widgets horizontally (Row) or vertically (Column).
- These are extremely flexible and can be customized with properties like alignment, spacing, and more.



Mobile App Development

Row and Column Widgets

- In Flutter, **Row** and **Column** are basic layout widgets used to arrange child widgets horizontally (Row) or vertically (Column).
- These are extremely flexible and can be customized with properties like alignment, spacing, and more.



A Row arranges its children horizontally in a single line.

- **mainAxisAlignment:** Controls how the children are aligned along the main axis (horizontal in the case of Row).
- **crossAxisAlignment:** Controls how the children are aligned along the cross axis (vertical in the case of Row).
- **children:** The list of widgets to display in the row.

Constructor Of Row Class:

```
Row(  
  {Key key,  
  MainAxisAlignment mainAxisAlignment: MainAxisAlignment.start,  
  MainAxisSize mainAxisSize: MainAxisSize.max,  
  CrossAxisAlignment crossAxisAlignment: CrossAxisAlignment.center,  
  TextDirection textDirection,  
  VerticalDirection verticalDirection: VerticalDirection.down,  
  TextBaseline textBaseline: TextBaseline.alphabetic,  
  List<Widget> children: const <Widget>[]  
})
```

body:

Center(

child: Row(mainAxisAlignment:

MainAxisAlignment.spaceEvenly, crossAxisAlignment:

CrossAxisAlignment.center,

children: [

Icon(Icons.star, color: Colors.red),

Icon(Icons.star, color: Colors.green),

Icon(Icons.star, color: Colors.blue),],

),),

Mobile App Development

Column Widget - Properties



A Column arranges its children vertically in a single column.

- **mainAxisAlignment:** Controls how the children are aligned along the main axis (vertical in the case of Column).
- **crossAxisAlignment:** Controls how the children are aligned along the cross axis (horizontal in the case of Column).
- **children:** The list of widgets to display in the column.

Constructor Of Column Class:

```
Column(  
{Key key,  
MainAxisAlignment mainAxisAlignment: MainAxisAlignment.start,  
MainAxisSize mainAxisSize: MainAxisSize.max,  
CrossAxisAlignment crossAxisAlignment: CrossAxisAlignment.center,  
TextDirection textDirection,  
VerticalDirection verticalDirection: VerticalDirection.down,  
TextBaseline textBaseline,  
List<Widget> children: const <Widget>[]  
} )
```


Mobile App Development

Column Widget - Example

body:

```
Center(  
  child: Column( mainAxisAlignment: MainAxisAlignment.center,  
    crossAxisAlignment: CrossAxisAlignment.center,  
    children:  
      [  
        Icon(Icons.star, color: Colors.red),  
        Icon(Icons.star, color: Colors.green),  
        Icon(Icons.star, color: Colors.blue),  
      ], ), ),
```

Mobile App Development

mainAxisAlignment Options:



- MainAxisAlignment.start: Aligns children at the start of the row/column.
- MainAxisAlignment.end: Aligns children at the end of the row/column.
- MainAxisAlignment.center: Aligns children in the center of the row/column.
- MainAxisAlignment.spaceBetween: Places free space between the children.
- MainAxisAlignment.spaceAround: Places free space around the children.
- MainAxisAlignment.spaceEvenly: Distributes free space evenly between and around the children.

- `CrossAxisAlignment.start`: Aligns children at the start along the cross axis.
- `CrossAxisAlignment.end`: Aligns children at the end along the cross axis.
- `CrossAxisAlignment.center`: Aligns children in the center along the cross axis.
- `CrossAxisAlignment.stretch`: Stretches children to fill the cross axis.

Both Row and Column are great for building flexible, responsive layouts in Flutter, and they can be nested or combined with other widgets for complex UIs.

Properties of Row and Column Widgets:

- **children:** This property takes in *List<Widget>*, that is a list of widgets to display inside the *Row* or the *Column* widget.
- **clipBehaviour:** This property holds *Clip* class as the object to decide whether the content on the *Row* or *Column* should be clipped or not.
- **crossAxisAlignment:** The *crossAxisAlignment* takes in *CrossAxisAlignment* *enum* as the object to how the children's widgets should be placed in *crossAxisAlignment*.
- For *Row* it is vertical and for *Column* it is horizontal.
- **direction:** This property holds as the *Axis* *enum* object to decide the direction used in the main axis. For *Row* and *Column*, it is fixed.

Properties of Row and Column Widgets:

- **mainAxisAlignment:** This property takes in *MainAxisAlignment* enum as the object to decide how the children widgets should be placed in *mainAxisAlignment*. For *Row* it is horizontal and for *Column* it is vertical.
- **mainAxisSize:** This property decides the size of main-axis by taking in *MainAxisSize* enum as the object.
- **runtimeType:** This property tells the run-time type of the *Row* or *Column* widget.
- **textBaseline:** This property is responsible for the alignment of the text in the *Row* or *Column* widget with respect to a baseline.
- **textDirection:** This property controls the text direction of the *Row* or *Column* widget, which can either be from left-to-right (by default) or right-to-left.
- **verticalDirection:** This property takes in *VerticalDirection* enum as the object to determine the order in which the children should be layered.

Mobile App Development

crossAxisAlignment Options:



Properties of Row and Column Widgets:

- We can align the content by using the following properties:
- *start*: Place the children from the starting of the row.
- *end*: Place the children at the end of the row.
- *center*: Place the children at the center of the row.
- *spaceBetween*: Place the space evenly between the children.
- *spaceAround*: Place the space evenly between the children and also half of that space before and after the first and last child.
- *spaceEvenly*: Place the space evenly between the children and also before and after the first and last child.

Mobile App Development

crossAxisAlignment Options:

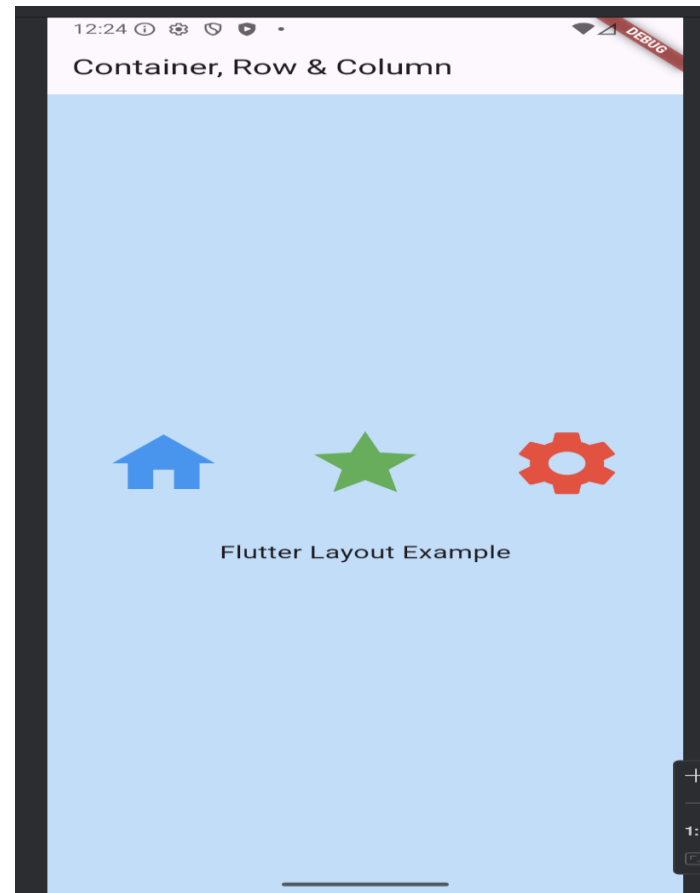
```
import 'package:flutter/material.dart';
void main() => runApp(MyApp());
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    double iconSize = 80.0;
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text("Container, Row &
Column")),
        body: Center(
          child: Container(
            padding: EdgeInsets.all(10),
            color: Colors.blue[100],
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
```

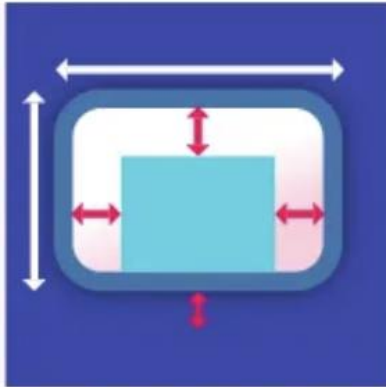
```
Row(
      mainAxisAlignment:
MainAxisAlignment.spaceAround,
      children: [
        Icon(Icons.home, color: Colors.blue, size:
iconSize),
        Icon(Icons.star, color: Colors.green, size:
iconSize),
        Icon(Icons.settings, color: Colors.red, size:
iconSize),
      ],
    ),
    SizedBox(height: 40),
    Text("Flutter Layout Example",
      style: TextStyle(fontSize: 18)) ],
    ), ), ), ),
  ); }
}
```

Mobile App Development

crossAxisAlignment Options:

Row and Column Widgets: output





Container

A convenience widget that combines common painting, positioning, and sizing widgets.



Row

Layout a list of child widgets in the horizontal direction.



Column

Layout a list of child widgets in the vertical direction.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Image Widget

Card Widget

App Icons

Dr. Annu Sharma

Department of Computer Applications

- In Flutter, the Image widget is used to display images in your app.
- Flutter provides different ways to load and display images from various sources like assets, network URLs, files, and memory.
- **Types of Image Sources:**
 - **Asset Images:** Images that are bundled with the app.
 - **Network Images:** Images fetched from a URL.
 - **File Images:** Images loaded from the device's file system.
 - **Memory Images:** Images loaded from bytes in memory.

Mobile App Development

Image Widget Syntax



- `Image.asset('assets/images/image_name.png');` // Asset image
- `Image.network('https://example.com/image.jpg');` // Network image
- `Image.file(File('path/to/image'));` // File image
- `Image.memory(bytes);` // Memory image

Mobile App Development

Properties of Image Widget



- **image:** The image source (from asset, network, file, or memory).
- **width:** Sets the width of the image.
- **height:** Sets the height of the image.
- **fit:** Determines how the image should be inscribed into the space (e.g., BoxFit.cover, BoxFit.contain).
- **alignment:** Aligns the image within its bounds (e.g., Alignment.center).
- **color:** Applies a color filter to the image.

Mobile App Development

Image Widget

```
Image.network(  
  
'https://cdn.pixabay.com/photo/2015/04/19/08/32/margu  
erite-729510_1280.jpg',  
    width: 500,  
    height: 300,  
),  
)
```



- The **Card** widget in Flutter is a convenient way to create a material design **card**—a visual element that groups related content and can contain images, text, and other widgets.
- It's commonly used to create a well-defined section with a drop shadow and rounded corners.

Example : Card Widget can be used to

- Displaying content in sections (like product listings, user profiles, etc.).
- Grouping related information (like settings or options).
- Creating interactive components (like buttons or actions within the card).

Card Widget Properties:

- **elevation**: Adds a shadow effect to give depth to the card. The higher the value, the more pronounced the shadow.
- **shape**: Defines the shape of the card. You can set custom shapes like rounded corners.
- **color**: Sets the background color of the card.
- **margin**: Controls the spacing around the card.
- **child**: The content of the card (can be any widget, like Column, Row, Text, Image, etc.).
- **clipBehavior**: Defines how the content inside the card is clipped, useful when using images.

Mobile App Development

Card Widget

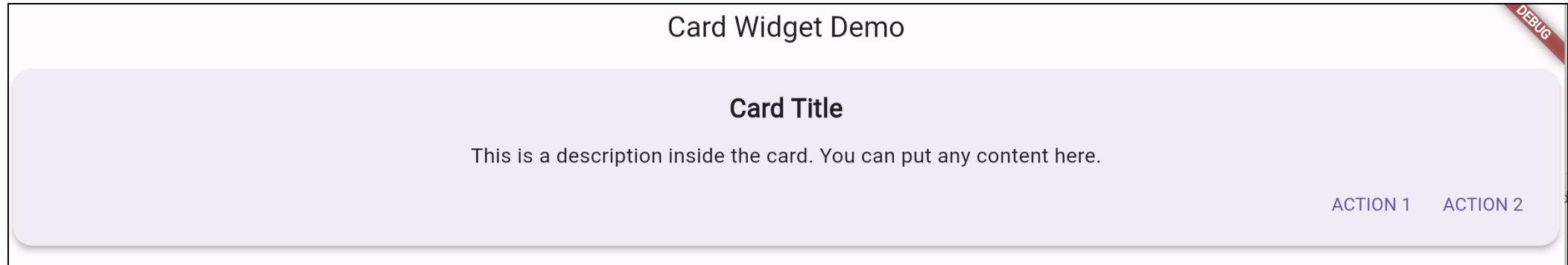


```
Card(  
  elevation: 4.0,  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(15.0),  
  ),  
  child: Padding(  
    padding: EdgeInsets.all(16.0),  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: [  
        Text(  
          'Card Title',  
          style: TextStyle(  
            fontSize: 20,  
            fontWeight: FontWeight.bold,  
          ),  
        ),  
        SizedBox(height: 10),
```

```
Text(  
  'This is a description inside the card. You can put any  
content here.',  
  style: TextStyle(fontSize: 16),  
),  
SizedBox(height: 10),  
Row(  
  mainAxisAlignment: MainAxisAlignment.end,  
  children: [  
    TextButton(  
      onPressed: () {},  
      child: Text('ACTION 1'),  
    ),  
    TextButton(  
      onPressed: () {},  
      child: Text('ACTION 2'),  
    ),  
  ], ),  
)
```

Mobile App Development

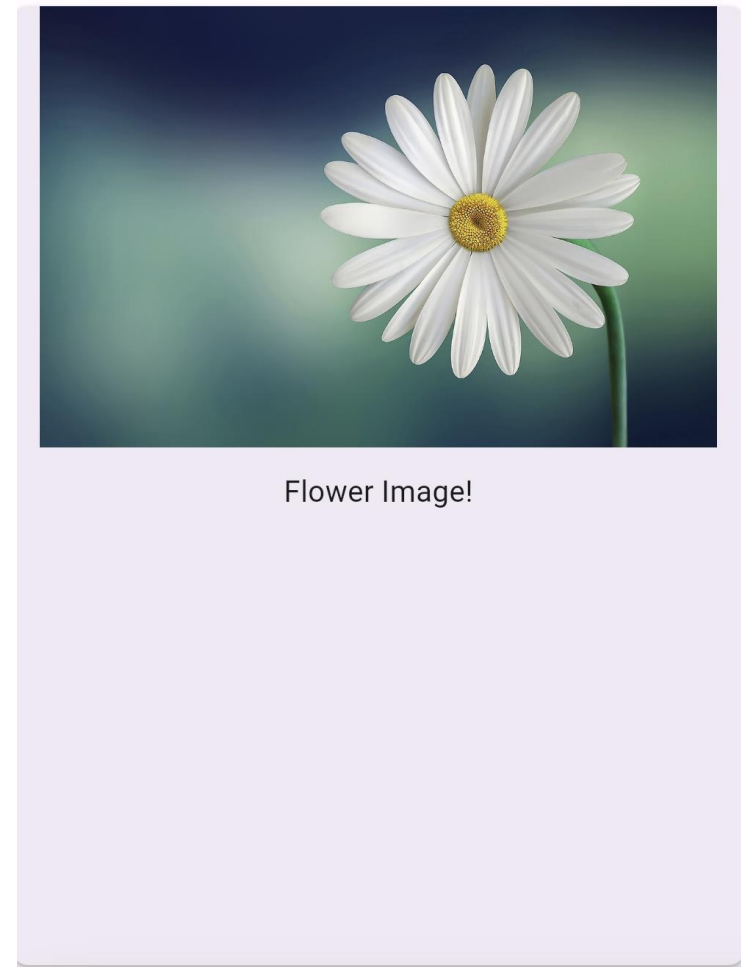
Card Widget - Output



Mobile App Development

Card Widget with an Image

```
Card(  
  clipBehavior: Clip.antiAlias,  
  elevation: 5.0,  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(12.0),  
  ),  
  child: Column(  
    children: [  
      Image.network('https://cdn.pixabay.com/photo/2015/04/  
/19/08/32/marguerite-729510_1280.jpg',  
        width: 500,  
        height: 300,),  
      Padding(  
        padding: EdgeInsets.all(16.0),  
        child: Text(  
          'Cute Kitten!',  
          style: TextStyle(fontSize: 20),  
        )),  
    ],  
  ),  
)
```



Mobile App Development

App Icons for iOS and Android Apps



- Designing app icons for iOS and Android is grounded in a combination of aesthetic principles, user experience considerations, and platform-specific guidelines.
- Here's a deeper look into the theoretical aspects behind creating effective app icons:
 1. Visual Identity and Branding
 2. Cognitive Load Theory
 3. Colour Theory
 4. User Experience (UX) Principles
 5. Platform Guidelines and Best Practices
 6. Technical Considerations
 7. Testing and Iteration



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOPs and Flutter Widgets Drawer

Dr. Annu Sharma

Department of Computer Applications

Drawer Widget

Introduction

The Drawer widget is used to create a sliding panel on the side of the screen, typically used for navigation. It provides a convenient way to access different parts of an app by swiping or tapping on a "hamburger" icon.

Drawer Widget

Introduction to Drawer Widget

- Drawer widget is used to provide access to different destinations and functionalities provided in your application.
- It is represented by three horizontal parallel lines on the upper end of the scaffold.
- It has a horizontal movement from the edge of the Scaffold that navigates the link to different routes in the flutter app.

Drawer Widget

Introduction to Drawer Widget

- Drawer widget is used to provide access to different destinations and functionalities provided in your application.
- It is represented by three horizontal parallel lines on the upper end of the scaffold.
- It has a horizontal movement from the edge of the Scaffold that navigates the link to different routes in the flutter app.

Drawer Widget

- The Navigating AppDrawer is divided into three sections namely header, body, and footer.
- The drawer's child that will be navigated to different destinations on getting tapped.
- All children of a Drawer widget are usually in ListView and initially, only the DrawerHeader is present in the UI which when tapped extends horizontally.

Drawer Widget

- **Key Parts**
- **Drawer Header:** The DrawerHeader widget is commonly used at the top of the drawer for branding or displaying user information. Customize it with styles or widgets.
- **ListTile:** Each ListTile represents a menu item. It includes an icon, a title, and an onTap function for navigation.

- The build method is where the UI is constructed.
- `build(BuildContext context) → Widget`: This method specifies the part of the UI rendered by the widget. This method is called when the Drawer widget is inserted into the tree in a given BuildContext and when the dependencies of the Drawer widget change.
- children widgets that will contain **ListTile**
- **ListTile**: This is a convenient way to organize the Drawer and the text Option.
- **leading**: `Icon(Icons.map)`: Places a Icon of map before the text, in the leading position (left side by default).

Drawer Widget Description

MaterialApp: Sets up the app with Material design and routes.

Scaffold: Provides a basic app layout with an AppBar and a body.

AppBar: A top bar with a title for the app.

ListView: A scrollable list container.

ListTile: A widget that creates a basic, consistent layout for each item in the list. Here, each ListTile has an icon (leading) and a title (title).

Drawer

```
import 'package:flutter/material.dart';  
void main() => runApp(MyApp());  
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: HomeScreen(),  
    );  
  }  
}
```

Drawer

```
class HomeScreen extends  
StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Drawer Example'),  
      ),  
    ),  
  },  
}
```

Drawer

```
drawer: Drawer(  
  child: ListView(  
    padding: EdgeInsets.zero,  
    children: <Widget>[  
      DrawerHeader(  
        decoration: BoxDecoration(  
          color: Colors.blue,  
        ),  
      ),  
    ],  
  ),  
)
```

```
child: Text(  
  'Drawer Header',  
  style: TextStyle(  
    color: Colors.white,  
    fontSize: 24,  
  ),  
)
```

Drawer

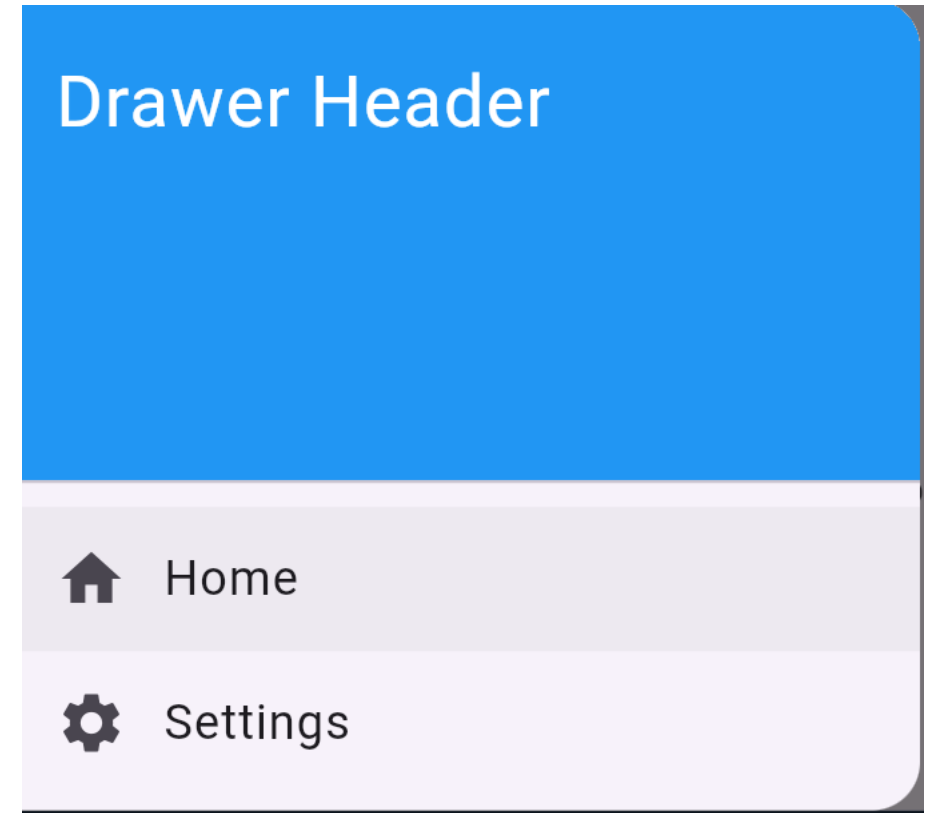
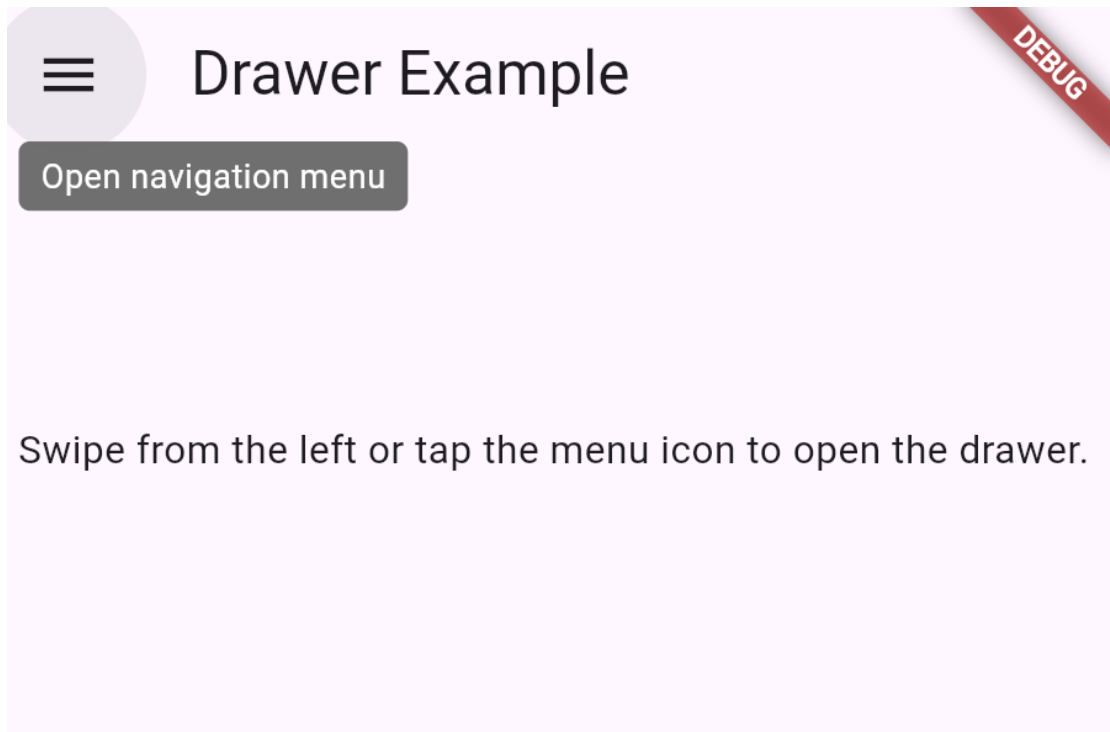
```
ListTile(  
  leading: Icon(Icons.home),  
  title: Text('Home'),  
  onTap: () {  
    // Navigate to Home  
    Navigator.pop(context);  
  },  
),
```

```
ListTile(  
  
  leading: Icon(Icons.settings),  
  title: Text('Settings'),  
  onTap: () {  
    // Navigate to Settings  
    Navigator.pop(context);  
  },  
), ], ), ),
```

Drawer

```
body: Center(  
  child: Text('Swipe from the left  
or tap the menu icon to open the  
drawer.'),  
),  
);  
}  
}
```

Output Drawer



Challenge

Create a Drawer Program in flutter with title as choice add three Icons of your choice in list.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science